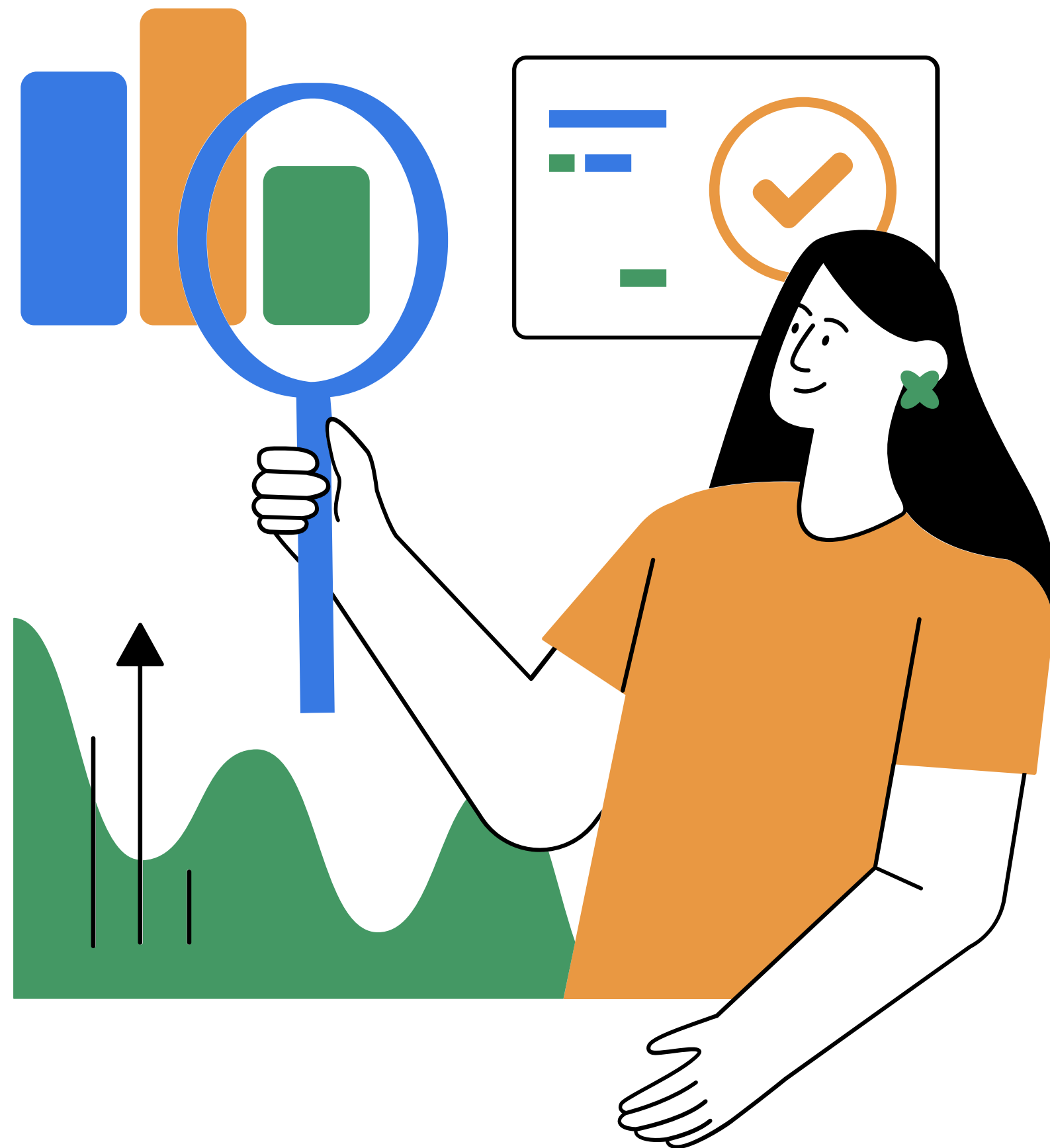


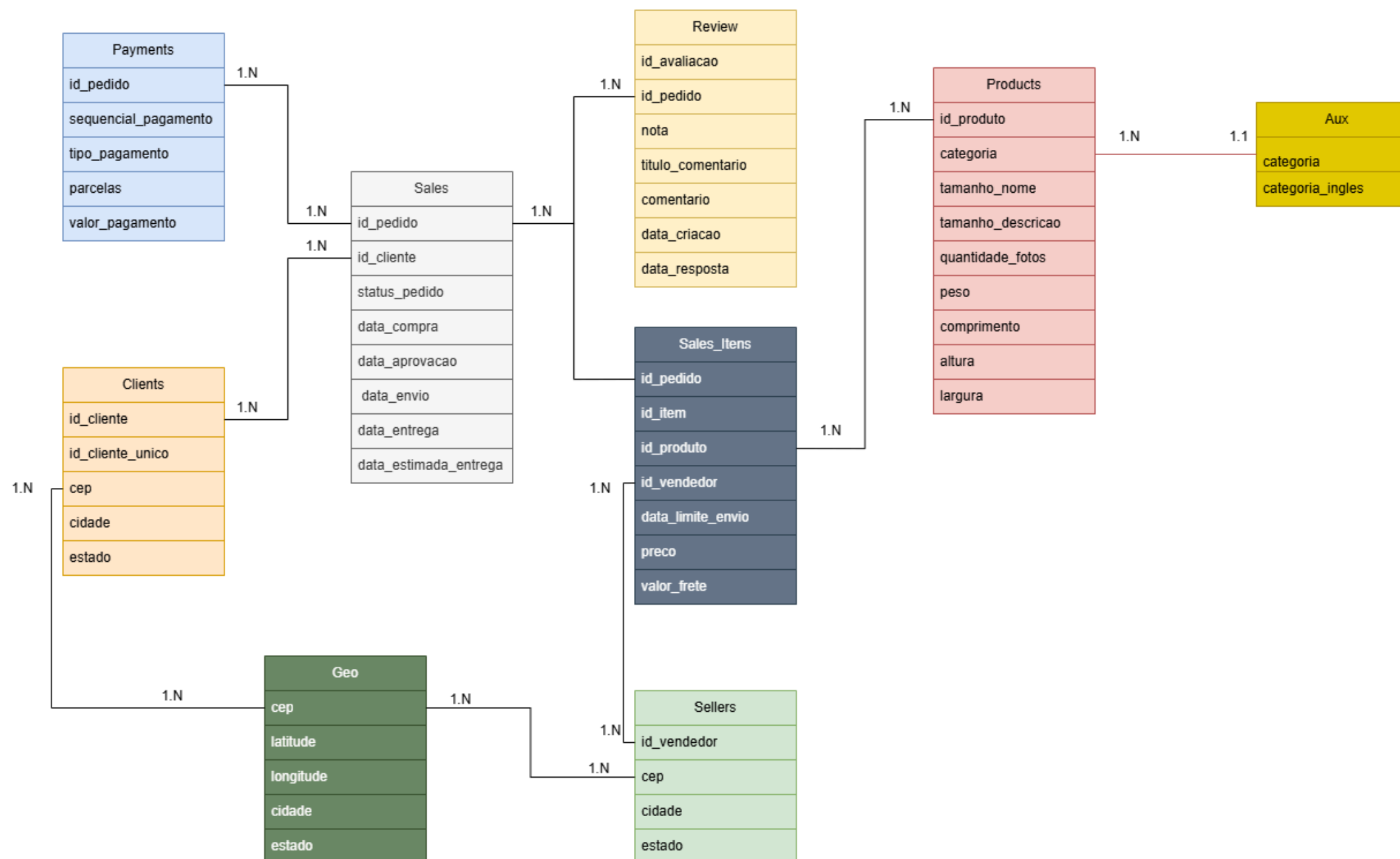
AWS O Poder das Palavras

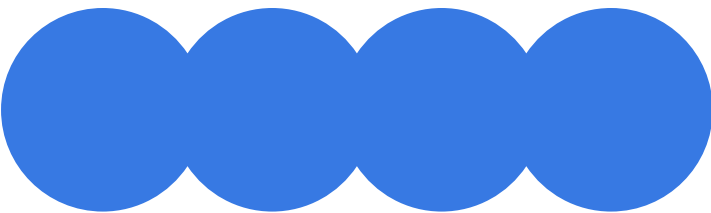




AWS - Poder das palavras

DER ←





Dicionário de Dados

Tabela	Pedidos				
Descrição	Representa os pedidos realizados pelos clientes no marketplace.				
Observações	Esta é a tabela central do modelo de dados.				
Campos					
Nome	Descrição	Tipo de Dado Arquitetura de Dados	Tipo de Dado no Python	Tamanho	Restrições de domínio (PK, FK, Not Null, Check, Default, Identity)
id_pedido:	Identificador único do pedido	Int*	Object	32	Chave Primária - PK / Not Null / Auto_Increment
id_cliente:	Identificador do cliente	Int	Object	32	Chave Estrangeira - FK / Unique / Not Null
status_pedido:	Status do pedido (entregue, enviado, cancelado)	VARCHAR(10)	Object	11	
data_compra:	Data da compra	DATETIME	Object		
data_aprovacao:	Data de aprovação do pagamento	DATETIME	Object		
data_envio:	Data de envio do pedido	DATETIME	Object		
data_entrega:	Data de entrega ao cliente	DATETIME	Object		
data_estimada_entrega:	Data estimada de entrega	DATETIME	Object		



```
normal_words = []
good_words = ["muito","bom","gosto", "legal", "bacana", "adorei", "otimo", "antes do prazo"]
great_words = []

negative_words = ["não recebi", "nada de chegar", "apesar", "atrazo", "atraso"]
bad_words = []

# IF score = 5
# if has normal words
# if has good words
# if has great words

# IF score = 4
# if has normal words
# if has good words
# if has great words

# IF score = 3
# if has normal words
# if has good words
# if has great words

# IF score = 2
# if has normal words
# if has good words
# if has great words

# IF score = 1
# if has normal words
# if has good words
# if has great words

positive_words = [
    "abençoado","abençoar","abertura","abnegação","abraço","abrigo","absoluto","abundância","ação","aceitação","aceitar","acessível","aclamação","aclamado","aconchegante","aconchegar","aconchego","acreditar","
    "balanceado","batalhador","batalhar","beijar","beijo","beldade","beleza","belo","bem-disposto","bem-educado","bem-estar","bem-humorado","bem-vindo","bênção","bendito","beneficência","beneficente","benefício","
    "calma","calor","camaradagem","capacidade","caridade","carinho","carinhoso","carismático","caritativo","cativar","cavalheiro","ceder","celebração","celestial","centrado","certeza","certo","céu","civildade"
]

positive_product_adjectives = [
    "ótimo", "excelente", "bom", "muito bom", "incrível",
    "maravilhoso", "perfeito", "fantástico", "espetacular", "impecável",
    "ideal", "eficiente", "eficaz", "confiável", "durável",
    "resistente", "robusto", "prático", "funcional", "versátil",
    "moderno", "inovador", "tecnológico", "avançado", "inteligente",
    "rápido". "ágil". "potente". "preciso". "estável".
```

```
def set_point_comment(text):
    return 0

def check_bad_comment(text):
    for i in negative_list:
        if i in text:
            return -1
    else:
        return 0

def check_good_comment(text):
    print(text)
    for i in positive_list:
        if i in text:
            return 1
    else:
        return 0

def get_comment_from_id(review_id):
    return df_review_clean[df_review_clean["review_id"] == review_id].review_comment_message.item()

def get_score_from_id(review_id):
    return df_review_clean[df_review_clean["review_id"] == review_id].review_score.item()
```




```

def sum_score_and_message(review_id):
    text = get_comment_from_id(review_id)
    score = get_score_from_id(review_id)

    bad_score = check_bad_comment(text)
    good_score = check_good_comment(text)

    print("good", good_score)
    print("bad", bad_score)

def decode_score(review_id):
    text = get_comment_from_id(review_id).lower()
    score = get_score_from_id(review_id)

    bad_score = check_bad_comment(text)
    good_score = check_good_comment(text)
    print(f"score: {score}. bad_score: {bad_score}. good_score:{good_score}")

    return score + (good_score + bad_score)

r = "0c9b6ca9b3beaf0fd5d621f63844ea01"

# good score bad word c1f198cdce8b992a9cddc3aaba11e689
#0c9b6ca9b3beaf0fd5d621f63844ea01
# bad score good word 7b66de927426b71a817aa36df5e8a0b3

score = decode_score(r)
score

print(get_comment_from_id("f7eb8792e67d53e01a224bfa9b6802b2"))

```

espero receber esta semana, o que não seria tão extravagante, porém...mais atrazo do que isso me faria desistir do baratheon.

score: 5. bad_score: -1. good_score:0

Cancelaram a minha compra um dia antes da entrega, liguei para a lannister e os atendentes não sabiam o que estava acontecendo, uma bagunça, agora estou esperando para ver se

```
from LeIA import SentimentIntensityAnalyzer

sid = SentimentIntensityAnalyzer ()

df_review_clean["review_comment_message"] = (
    df_review_clean["review_comment_message"].fillna("")
)

def calculate_comment_portuguese(text):
    if text.strip () == "":
        return 0.0
    return sid.polarity_scores(text)["compound"]

def get_sentiment_category_from_point(point):
    if point <= -0.6:
        return "Muito negativo"
    elif point > -0.6 and point <= -0.2:
        return "Negativo"
    elif point > -0.6 and point < -0.2:
        return "Neutro"
    elif point > -0.6 and point < -0.2:
        return "Positivo"
    else:
        return "Muito positivo"

get_sentiment_category_from_point(calculate_comment_portuguese("Entrega antes do prazo, recomendo essa loja."))

'Muito positivo'
```

Thank You

- ANA
- JUSTINO

